CERTIK

Wemix 180 GovImp

Security Assessment

CertiK Assessed on Jun 9th, 2026





CertiK Assessed on Jun 9th, 2026

Wemix 180 GovImp

The security assessment was prepared by CertiK.

Executive Summary

TYPES	ECOSYSTEM	METHODS
Governance	WEMIX	Manual Review, Static Analysis, Testnet Deployment

LANGUAGE	TIMELINE
Solidity	Preliminary comments published on 06/03/2026 Final report published on 06/09/2026

Vulnerability Summary

4	4	0	0	0
Total Findings	Resolved	Partially Resolved	Acknowledged	Declined

0 Centralization		Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.
0 Critical		Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.
0 Major		Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.
0 Medium		Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.
4 Minor	4 Resolved	Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.
0 Informational		Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | WEMIX 180 GOVIMP

Audit Summary

Executive Summary

Vulnerability Summary

Codebase

Audit Scope

Approach & Methods

Findings

W1G-01 : Queued Add-Member Proposals Can Bypass Node Uniqueness At Execution

W1G-02 : Stale Removal Proposals Can Remove The Sole Remaining Member

W1G-03 : Migration And Reinitialization Paths Fail To Preserve Node Uniqueness Markers

W1G-04 : Self-Finalized Member Updates Can Corrupt Staker And Voter Index Invariants

Appendix

Disclaimer

CODEBASE | WEMIX 180 GOVIMP

Repository

https://github.com/WEMADE-Ext/waffle/

Commit

6f1937591f469cfaf11595c25859981e9587cfe7
ae77ec47cf818f4117fa46f7ef6d6ace734244d6

AUDIT SCOPE | WEMIX 180 GOVIMP

CertiKProject/certik-audit-projects

GovImp.sol

APPROACH & METHODS | WEMIX 180 GOVIMP

This audit was conducted for Wemix to evaluate the security and correctness of the smart contracts associated with the Wemix 180 GovImp project. The assessment included a comprehensive review of the in-scope smart contracts. The audit was performed using a combination of Manual Review, Static Analysis, and Testnet Deployment.

The review process emphasized the following areas:

- Architecture review and threat modeling to understand systemic risks and identify design-level flaws.
- Identification of vulnerabilities through both common and edge-case attack vectors.
- Manual verification of contract logic to ensure alignment with intended design and business requirements.
- Dynamic testing to validate runtime behavior and assess execution risks.
- Assessment of code quality and maintainability, including adherence to current best practices and industry standards.

The audit resulted in findings categorized across multiple severity levels, from informational to critical. To enhance the project's security and long-term robustness, we recommend addressing the identified issues and considering the following general improvements:

- Improve code readability and maintainability by adopting a clean architectural pattern and modular design.
- Strengthen testing coverage, including unit and integration tests for key functionalities and edge cases.
- Maintain meaningful inline comments and documentations.
- Implement clear and transparent documentation for privileged roles and sensitive protocol operations.
- Regularly review and simulate contract behavior against newly emerging attack vectors.

FINDINGS | WEMIX 180 GOVIMP

4	0	0	0	0	4	0
Total Findings	Critical	Centralization	Major	Medium	Minor	Informational

This report has been prepared for Wemix to identify potential vulnerabilities and security issues within the reviewed codebase. During the course of the audit, a total of 4 issues were identified. Leveraging a combination of Manual Review, Static Analysis & Testnet Deployment the following findings were uncovered:

ID	Title	Category	Severity	Status
W1G-01	Queued Add-Member Proposals Can Bypass Node Uniqueness At Execution	Volatile Code	Minor	Resolved
W1G-02	Stale Removal Proposals Can Remove The Sole Remaining Member	Volatile Code	Minor	Resolved
W1G-03	Migration And Reinitialization Paths Fail To Preserve Node Uniqueness Markers	Logical Issue	Minor	Resolved
W1G-04	Self-Finalized Member Updates Can Corrupt Staker And Voter Index Invariants	Logical Issue	Minor	Resolved

W1G-01 Queued Add-Member Proposals Can Bypass Node Uniqueness At Execution

Category	Severity	Location	Status
Volatile Code	Minor	GovImp.sol: 526	Resolved

Description

Node name, enode, and IP:port uniqueness is checked when an add-member proposal is created, but `addMember` does not re-check those fields when the proposal is executed. The contract already revalidates other dynamic conditions at execution time, such as membership, reward address, staking bounds, and available balance, but omits the node uniqueness invariant.

Two add-member proposals can therefore be queued with the same node metadata while no accepted member currently uses it. After the first proposal is accepted, the second can still be accepted and will write duplicate node metadata, breaking the contract's explicit uniqueness guarantees for validator identity.

Recommendation

Call `checkNodeInfoAdd(name, enode, ip, port)` inside `addMember` immediately before writing the new node, and reject the ballot as not applicable if the metadata is no longer unique.

Alleviation

The client revised the code and resolved this issue in commit : ae77ec47cf818f4117fa46f7ef6d6ace734244d6

W1G-02 Stale Removal Proposals Can Remove The Sole Remaining Member

Category	Severity	Location	Status
Volatile Code	Minor	GovImp.sol: 600	Resolved

Description

`addProposalToRemoveMember` prevents creating a removal proposal when only one member exists, but `removeMember` does not enforce the same invariant at execution. Because proposals can be queued while more than one member exists, later removals can reduce `memberLength` before an older removal proposal is executed.

If the stale proposal is then accepted, `removeMember` decrements `memberLength` to zero. With no remaining governance members, voting, upgrades, and environment changes become unrecoverable through the normal governance path.

Recommendation

Re-check `getMemberLength() > 1` inside `removeMember` before mutating member state. If the invariant no longer holds, emit `NotApplicable` and reject the ballot.

Alleviation

The client revised the code and resolved this issue in commit : ae77ec47cf818f4117fa46f7ef6d6ace734244d6

W1G-03 Migration And Reinitialization Paths Fail To Preserve Node Uniqueness Markers

Category	Severity	Location	Status
Logical Issue	Minor	GovImp.sol: 1026, 1068	Resolved

Description

`reInit` iterates from index 0 while member and node storage are consistently 1-indexed elsewhere. As a result it marks the empty `nodes[0]` entry and skips the last real node, leaving that node's uniqueness markers unset during a backfill.

`migrateFromLegacy` also calls `checkNodeInfoChange` with the candidate node passed as both the new node and the existing node. That helper treats unchanged values as exempt from duplicate checks, so this call does not actually reject duplicate node metadata from the legacy source. Both paths can leave the duplicate-check mappings inconsistent with the actual node table.

Recommendation

Change `reInit` to iterate from `1` through `getMemberLength()` inclusive. During migration, validate each imported node against already-imported markers with `checkNodeInfoAdd` or direct marker checks before setting the markers.

Alleviation

The client revised the code and resolved this issue in commit : ae77ec47cf818f4117fa46f7ef6d6ace734244d6

W1G-04 Self-Finalized Member Updates Can Corrupt Staker And Voter Index Invariants

Category	Severity	Location	Status
Logical Issue	Minor	GovImp.sol: 311	Resolved

Description

The self-update branch in `addProposalToChangeMember` auto-starts and auto-finalizes a `MemberChange` ballot when `msg.sender == oldStaker && oldStaker == newInfo.staker`. That fast path reuses validation that only checks generic member/reward status, but it does not enforce the staker and voter reverse-index invariants that `changeMember` later assumes before writing state.

First, `checkChangeMember` accepts any `oldStaker` that satisfies `isMember(oldStaker)`, then immediately uses `stakerIdx[oldStaker]` and `nodeIdxFromMember[oldStaker]` as if `oldStaker` were guaranteed to be a real staker key. Because voters are also treated as members, a voter-only address can satisfy the gate while both indexes resolve to `0`. Through the self-update path, that address can validate against slot `0` and make `changeMember` write node metadata and role bookkeeping into index `0`, including poisoning `checkNodeName`, `checkNodeEnode`, and `checkNodeIpPort` with arbitrary values.

Second, the same self-update path can reuse a voter address that is already assigned to another member. Proposal-time validation rejects `newInfo.voter` only when it is already a member or reward address, not when it is already an active voter. `checkChangeMember` repeats the same omission, and `changeMember` blindly executes `voterIdx[newVoter] = memberIdx`. The latest self-update can therefore steal the reverse voter mapping from the previous member, leaving stale `voters[]` state and causing later `getStakerAddr` resolution to point that voter at the attacker's member.

Recommendation

Apply staker- and voter-specific invariant checks before allowing self-finalized member updates to write state. Require `oldStaker` to be an actual staker entry, enforce `stakerIdx[oldStaker] != 0` and `nodeIdxFromMember[oldStaker] != 0`, and reject any `newInfo.voter` that is already present in `voterIdx` unless it is the current voter of `oldStaker`. Repeat these checks inside `checkChangeMember` and immediately before updating `stakerIdx`, `voterIdx`, and `nodeIdxFromMember` so duplicate or index-zero assignments cannot be committed even if proposal validation is bypassed.

Alleviation

The client revised the code and resolved this issue in commit : ae77ec47cf818f4117fa46f7ef6d6ace734244d6

APPENDIX | WEMIX 180 GOVIMP

Finding Categories

Categories	Description
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevate Your Web3 Journey

CertiK is the largest Web3 security platform combining formal verification with audits and comprehensive security solutions.